LAB MANUAL

ZCOER
PUNE

Software Lab III- DMWL(GroupA)

T.Y.B.Tech.

(With effect from- AcademicYear2025- 26)

Department

of

Artificial Intelligence & Machine Learning

Zeal Education Society's

ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE-41

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)`

NAACA credited with A+ Grade/ISO21001:2018

VISION AND MISSION

Institute:

Vision	To be a premier institute in technical education by imparting academic excellence, research, social and entrepreneurial attitude.
Mission	1. To achieve academic excellence through innovative teaching and learning process. 2. To imbibe the research culture for addressing industry and societal needs. 3. To inculcate social attitude through community engagement initiatives. 4. To provide conducive environment for building the entrepreneurial skills

Department:

Vision	**To produce competent artificial intelligence and Machine Learning professionals to serve the needs of the society.**
Mission	**M1:** To impart quality, skill-based and value-based education to the students in the field of Artificial Intelligence and Machine Learning **M2:** To identify industrial requirements and enhance the students' expertise. **M3:** To build an ecosystem that will have an ethical impact on the society.

SYLLABUS

Savitribai Phule Pune University, Pune
Third Year of Artificial Intelligence and Machine Learning (2020 Course)
318553: Data Mining & Warehousing

Teaching Scheme:	Credit Scheme:	Examination Scheme:
Theory (TH) : 3 hrs/week	03 Credits	Mid_Semester : 30 Marks End_Semester : 70 Marks

Prerequisite Courses: Database Management Systems

Course Objectives:

- Learn how to build a data warehouse and query it (using open source tools like Pentaho Data Integration Tool, Pentaho Business Analytics).
- Learn to perform data mining tasks using a data mining toolkit (such as open source WEKA).
- Understand the data sets and data preprocessing.
- Demonstrate the working of algorithms for data mining tasks such association rule mining, classification, clustering and regression.
- Exercise the data mining techniques with varied input values for different parameters.
- To obtain Practical Experience Working with all real data sets.
- Emphasize hands-on experience working with all real data sets.

Course Outcomes:

On completion of the course, students will be able to–

CO1: Ability to understand the various kinds of tools.

CO2: Apply frequent pattern and association rule mining techniques for data analysis

CO3: Apply appropriate classification and clustering techniques for data analysis

CO4: Apply frequent pattern and association rule mining techniques for data analysis & Study Warehouse with design and Components.

CO5: Apply suitable pre-processing and visualization techniques for data analysis

CO6: Design a Data warehouse system and perform business analysis with OLAP tools.

COURSE CONTENTS

Unit I	INTRODUCTION TO DATA MINING	(07 hrs)

Definition of data mining- Data Mining Techniques – Issues – applications- Data Objects and attribute types-knowledge discovery Process, Data Mining Functionalities, Classification of Data Mining Systems , Statistical description of data, Data Preprocessing – Cleaning, Integration, Reduction, Transformation and discretization, Data Visualization, Data similarity and dissimilarity measures. Data Mining Task Primitives, Mining Frequent Patterns, Associations, Market Basket Analysis, Apriori Algorithm, Association rules from frequent item set, Text Mining and Web Mining.

Mapping of Course Outcomes for Unit I	**CO1**	
Unit II	**CLASSIFICATION AND PREDICTION**	**(07 hrs)**

Introduction, Classification by Decision Tree Induction, Attribute selection measures, Bayesian Classification , Bayes Theorem ,Naïve Bayesian Classification , Rule-Based Classification, If then rules for classification, Rule Extraction from decision tree, Classification by Back propagation, Support Vector Machines. Mining **Data Mining-frequent Pattern Analysis:** Frequent Patterns, Associations and Correlations – Mining Methods- Pattern Evaluation Method – Pattern Mining in Multilevel, Multi-Dimensional Space – Constraint Based Frequent Pattern Mining, Classification using Frequent Patterns.		
Case Study	**WEKA Tool**	
Mapping of Course Outcomes for Unit II	**CO2**	
Unit III	**CLUSTER ANALYSIS AND BUSINESS INTELLIGENCE**	**(07 hrs)**
Introduction to Cluster Analysis-Types, Categorization of Major Clustering Methods, Clustering techniques: - Partitioning (k-means, k-means++, Mini-Batch k-means, k-medoids), Hierarchical (Agglomerative and Divisive), Density based (DBSCAN), Grid Based Methods, Outlier analysis-outlier detection methods, **Business Intelligence:** Introduction to Data, Information, and Knowledge, Design and implementation aspect of OLTP, Introduction to Business Intelligence and Business Models, BI Definitions & Concepts, Business Applications of BI, Role of DW in BI, BI system components, Components of Data Warehouse Architectures.		
Case Study	**Learn Different ETL Tools**	
Mapping of Course Outcomes for Unit III	**CO3,CO4**	
Unit IV	**INTRODUCTION TO DATA WAREHOUSING**	**(07 hrs)**
Introduction to Decision Support System, Need for data warehousing, Operational & informational data, Data Warehouse definition and characteristics, Data Warehouse Architecture. **Warehouse Design:** The Process of Data Warehouse Design, A Three-Tier Data Warehouse Architecture, Conceptual modelling of data warehouse, Differences between operational database and data warehouse, Data warehouse implementation, Data marts, Components of data warehouse, Need for data warehousing ,Trends in data warehousing		
Mapping of Course Outcomes for Unit IV	**CO4**	
Unit V	**DATA WAREHOUSE COMPONENTS**	**(07 hrs)**
Architectural components: ETL Process, Data Preprocessing: Why Preprocess Data? Data Life Cycle, Data Cleaning Techniques, Data Integration and Transformation, Data Reduction strategies overview, Discretization and Concept Hierarchy Generation for numerical data techniques binning, histogram analysis, For categorical data techniques concept hierarchies, Significant role of metadata, Data warehouse applications and usage.		
Case Study	**Graph Mining and Social Network Analysis**	

Mapping of Course Outcomes for Unit V	**CO5**	
Unit VI	**OLAP IN THE DATA WAREHOUSE**	**(07 hrs)**
A Multidimensional Data Model, Schemas for Multidimensional Databases: Stars, Snowflakes, and Constellations Measures, Concept Hierarchies, OLAP Operations in the Multidimensional Data Mo Need for OLAP, OLAP tools, Types-ROLAP, MOLAP, HOLAP.		
Case Study	**Mining Spatial, Multimedia, Text and Web Data**	
Mapping of Course Outcomes for Unit VI	**CO6**	
Text Books:		
1. Data Mining Concepts and Techniques by Han, Kamber, Morgan Kaufmann , MK publication. 2. Data Mining: Concepts and Techniques by Margaret Dunham, Morgan Kaufmann Publication. 3. Data Warehousing Fundamentals by Paul Punnian, John Wiley Publication. 4. Data Warehousing, Data Mining and OLAP by Alex Berson, S.J. Smith, Tata McGraw Hill		
Reference Books:		
1. The Data Warehouse Lifecycle Toolkit by Ralph Kimball, John Wiley 2. Business Intelligence: A Managerial Approach (2nd Ed,) Turba. N, Sharda, Delen, King.Wil Publication		

List of Experiments:

Any seven Assignments are compulsory*

Assignment No 1: Build Data Warehouse and Explore WEKA

Assignment No 2: Perform data preprocessing tasks and Demonstrate performing association rule mining on data sets

Assignment No 3: Demonstration of classification rule process on WEKA data-set using Naive Bayes algorithm.

Assignment No 4: Implementation of OLAP operations

Assignment No 5: Demonstrate performing Regression on data sets

Assignment No 6: Demonstration of clustering rule process on data-set iris.arff using simple k-means

Assignment No 7: Demonstration of any ETL tool

Assignment No 8: Write a program of Apriori algorithm using any programming language.

Assignment No 9: Case Study on Text Mining or any commercial application.

Text Books:

1. Daniel Minoli, "Building the Internet of Things with IPv6 and MIPv6: The Evolving World of M2M Communications", ISBN: 978-1-118-47347-4, Willy Publications
2. Bernd Scholz-Reiter, Florian 2. Michahelles, "Architecting the Internet of Things", ISBN 978-3-642-19156-5 e-ISBN 978-3-642-19157-2, Springer
3. Data Mining: Concepts and Techniques by Margaret Dunham, Morgan Kaufmann Publication.
4. Data Warehousing Fundamentals by Paul Punnian, John Wiley Publication.

Reference Books:
1. Data Warehousing, Data Mining and OLAP by Alex Berson, S.J. Smith, Tata McGraw Hill
E-Resources: 1. https:// www.tutorialspoint.com/weka

DO's AND DON'Ts

Do's:

- Please follow instructions precisely as instructed by your Course Teacher.
- Always perform the experiment or activity precisely as directed by Course Teacher.
- Before performing experiment, read Lab manual carefully.
- Follow all written and verbal instructions carefully.
- Remove your shoes before you enter the lab.
- Always keep quiet. Be considerate to other lab users.
- Report any problems with the computer to the Course Teacher in charge.
- Shut down computers properly.
- Switch off power supply of devices, tubes& fans whenever not necessary.
- Arrange your own chair properly before leaving the lab.

Don'ts:

- The use of personal audio or video equipment is prohibited in the laboratory.
- Don't use mobile phones during lab hours.
- Do not bring any food or drinks in the lab.
- Do not touch any part of computer with we hand.
- Do not hit the keys on the computer too hard.
- Don't damage, remove or disconnect labels, parts, cables or equipment.
- Do not install or download any software or modify or delete any system files on any lab computers.

INDEX

Sr No.	Title	Page No.	Conducted Date	Submission Date	Sign
1	Build Data Warehouse and Explore WEKA				
2	Perform data preprocessing tasks and Demonstrate performing association rule mining on data sets				
3	Demonstration of classification rule process on WEKA data-set using Naive Bayes algorithm				
4	Implementation of OLAP operations				
5	Demonstration of any ETL tool				
6	Write a program of Apriori algorithm using any programming language.				
7	Case Study on Text Mining or any commercial application				

CERTIFICATE

This is to certified that Mr./Miss.____

of class __________Division ______Roll no. __________has completed Experiments / Practical

work / Assignments satisfactorily in the subject of ____________________________during

the Semester - ______of the academic year ____________.

Prof. Manisha A. Devgunde
Course Faculty

Dr. Dipali Shende
Head of Department

Experiment No.:1

Build Data Warehouse and Explore WEKA

Date of Performance:

Particulars	**Marks**
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
Signature of Course Teacher	

Expirement:1

Objective:

- Install Weka Work Bench/ Notepad Editor.

Theory:

WEKA - an opensource software provides tools for data preprocessing, implementation of several Machine Learning algorithms, and visualization tools so that you can develop machine learning techniques and apply them to real-world data mining problems.

Requirements:

- WEKA 3.8.6 (latest version) Download from
- Notepad Text Editor

Installation Steps:

Step 1: Installing Weka

1. Download Weka from official site
2. Click "Install Now" and wait for the installation to complete.
3. If Weka is installed correctly, it will display the installed version.

You will see the following screen on successful installation.

Writing and Running the Data Set

Step 1:
Create the data set on Notepad

session 42 rather than project | Untitled | A2- | @relation market.arff

File Edit View

```
@relation Market
@attribute id numeric
@attribute items string {Bread,Butter,Jam,Milk,juice,curd}
@data
1,Bread,Butter,Jam,Milk
2,Bread,Butter,Milk
3,Bread,juice,curd
4,Bread,Milk,Juice
5,Butter,Milk,Juice
```

Explore the Weka
Step 1: Preprocess the dataset
Step2: Two ways are to select the preprocess data
1.Open File-Windows C:-Program Files-WEKA 3.8.6-data-select dataset
2.Open-Desktop-Own Folder- arff dataset

Step 2: Select the operational tabs
Tab 1:Classify
Tab 2:Cluster
Tab3:Associate
Tab4:Select Attribute
Tab5:Visualize

Step3:Selection of Filters
1.Supervised Filters
2.Unsupervised Filters

Step4: Choose Algorithm
1.Apriori
2.Frequent Pattern Growth
3. Naïve Bays Algorithm

Step 5: Start
Step 6: Visualize Output

Experiment No.:2

Perform data preprocessing tasks and demonstrate performing association rule mining on data sets

Date of Performance:

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
SignatureofCourse Teacher	

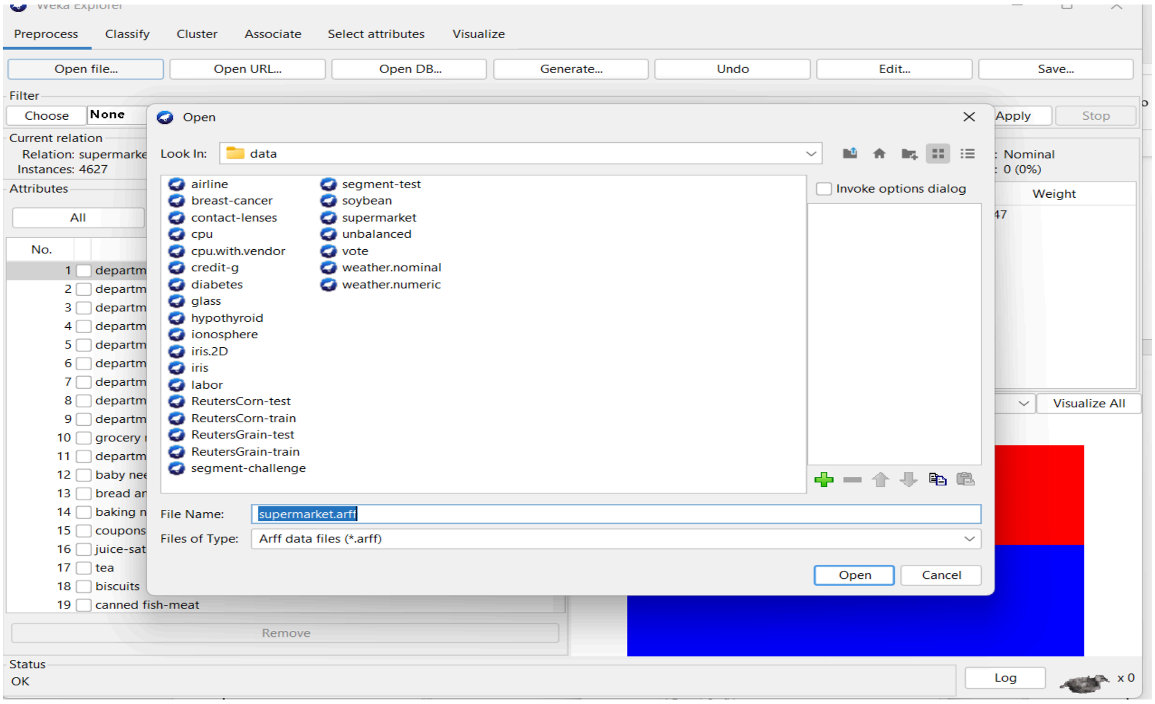
Objective:

Perform the preprocessing on the dataset to check the association algorithms are working.

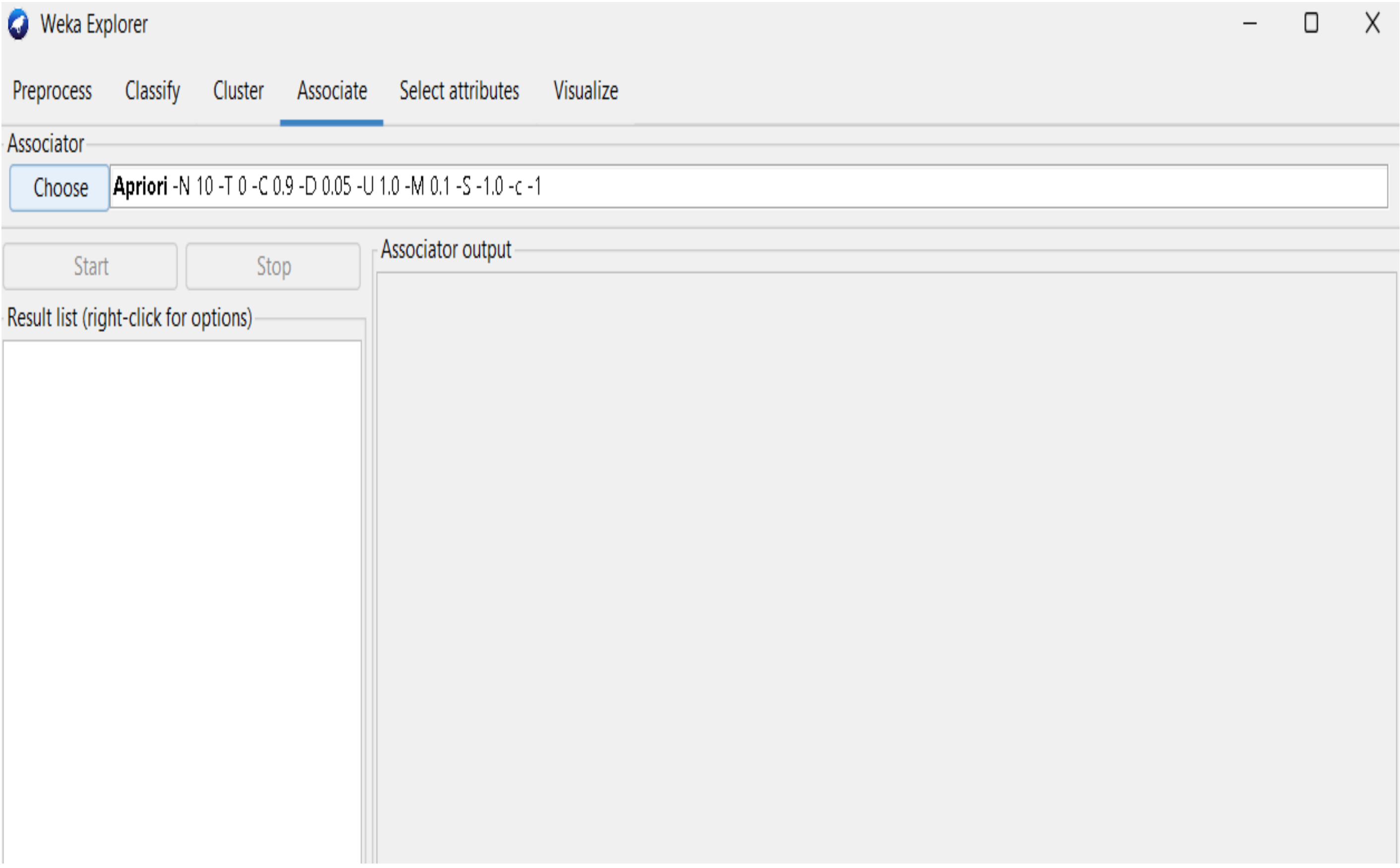
Procedure:

For Apriori Algorithm

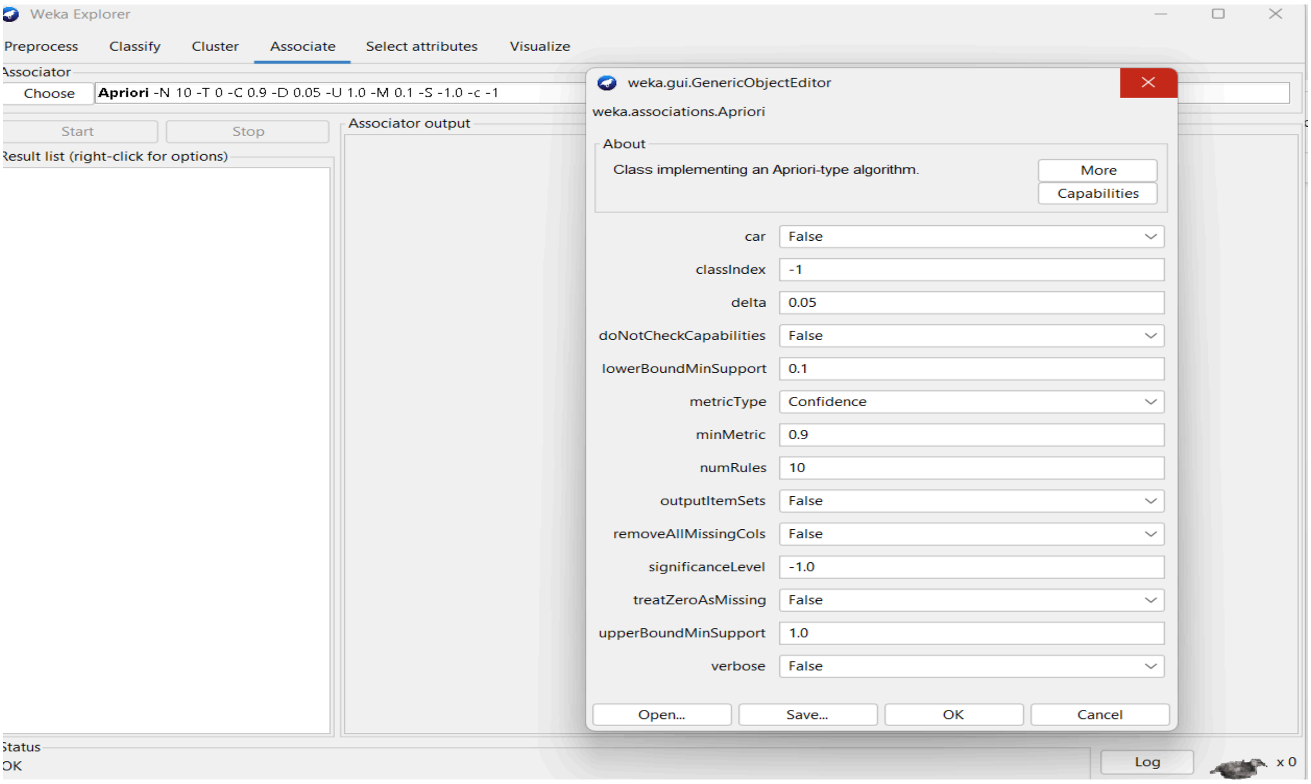
Step 1: Select the appropriate dataset from the source file.



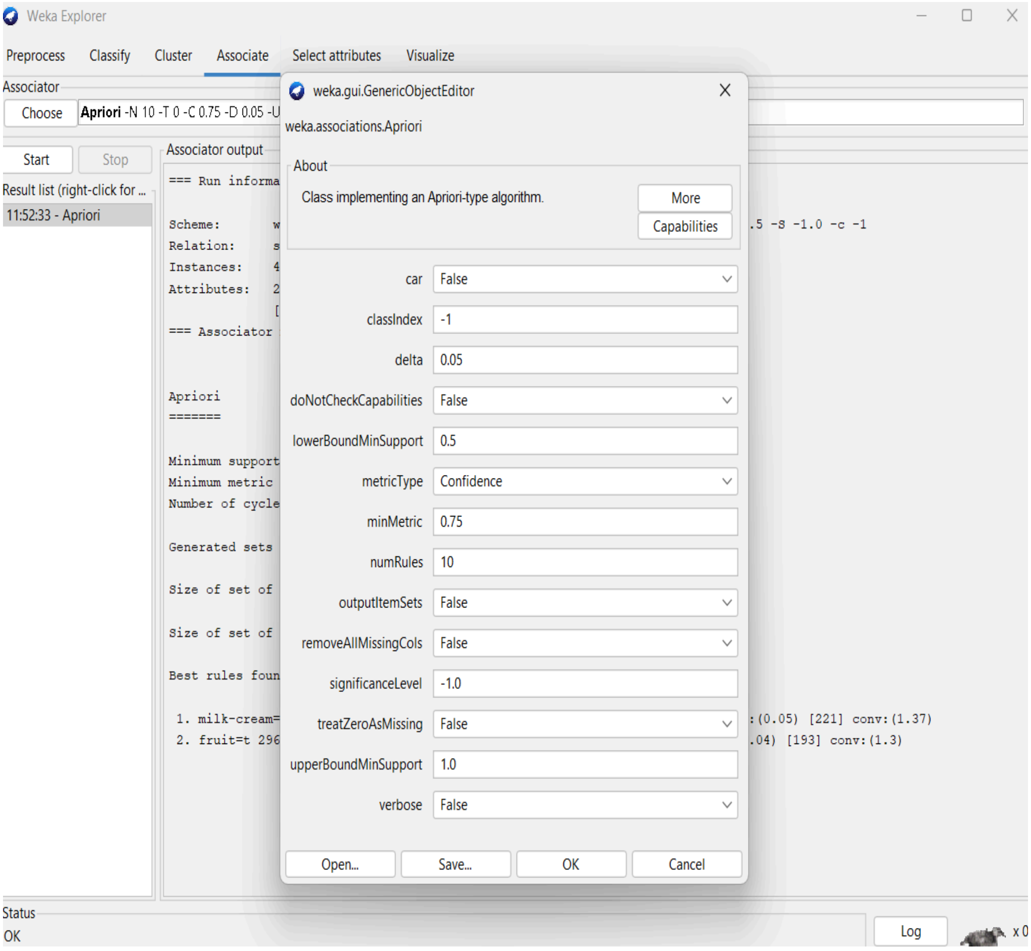
Step 2: Verify the dataset is loaded or not and also check related attributes and tables inside the dataset



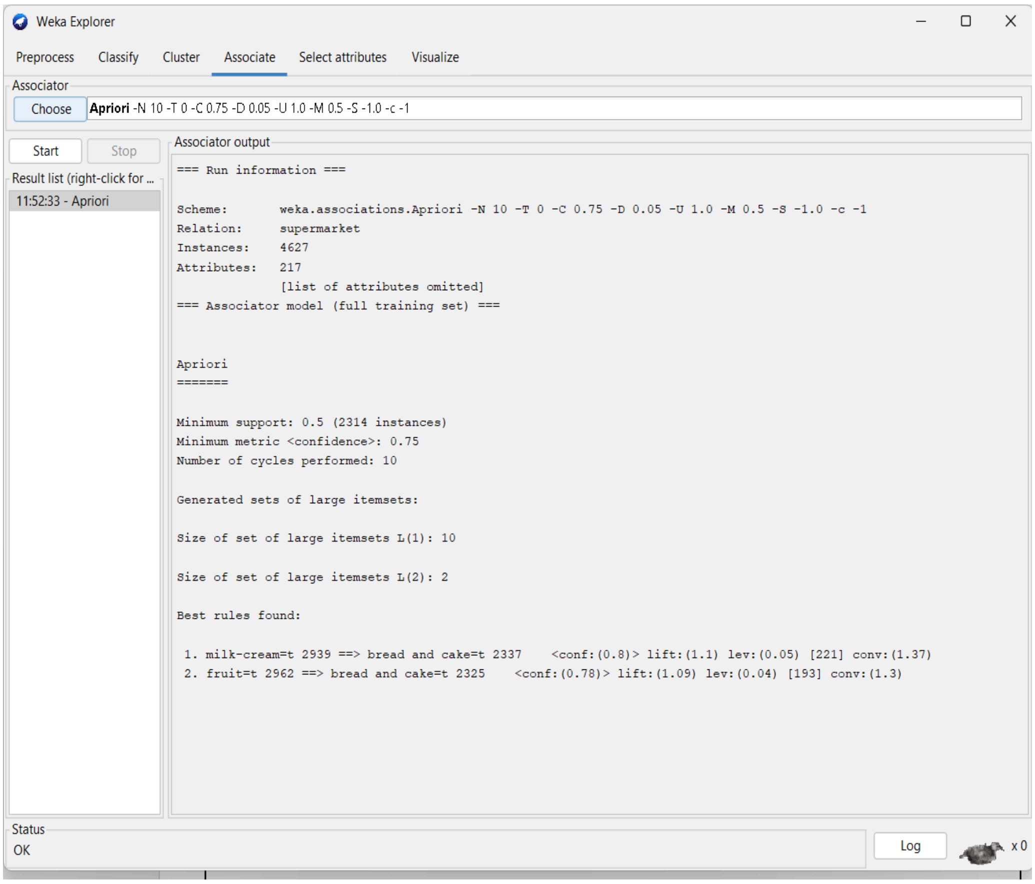
Step3: Select the preprocessing algorithm Apriori from the Weka predefined libraries.



Step4: Check/Select properties from WEKA choose tab and set the classes standards and values that is applicable for the preprocessing of algorithm.

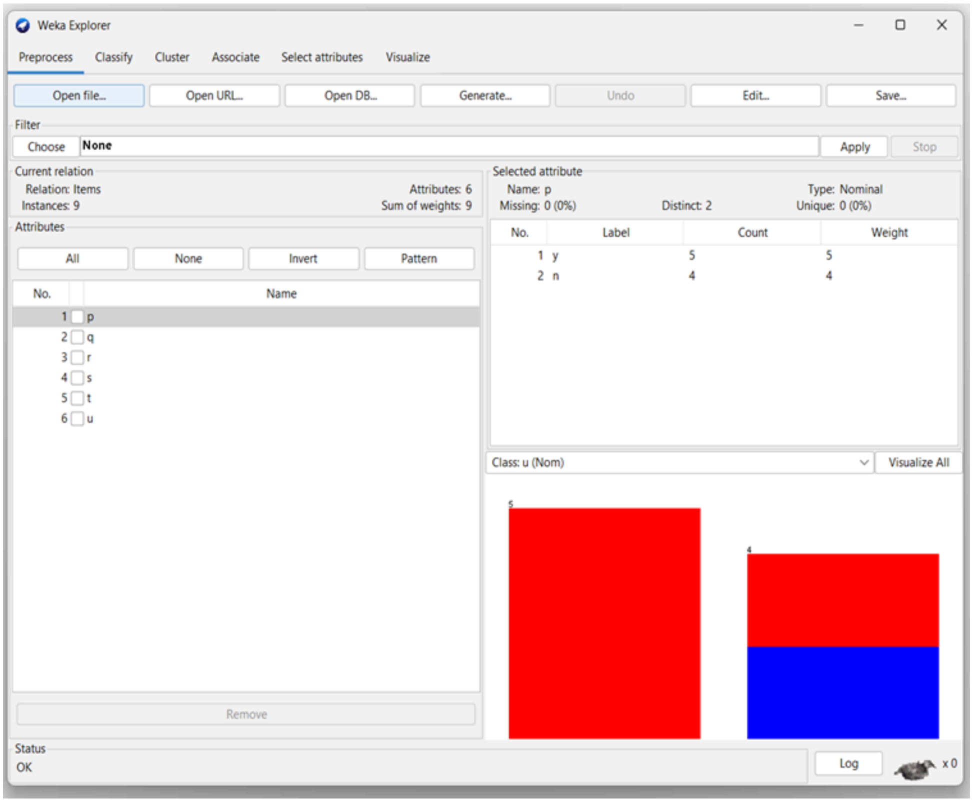


Step5: Click on start button and check for accuracy on output screen

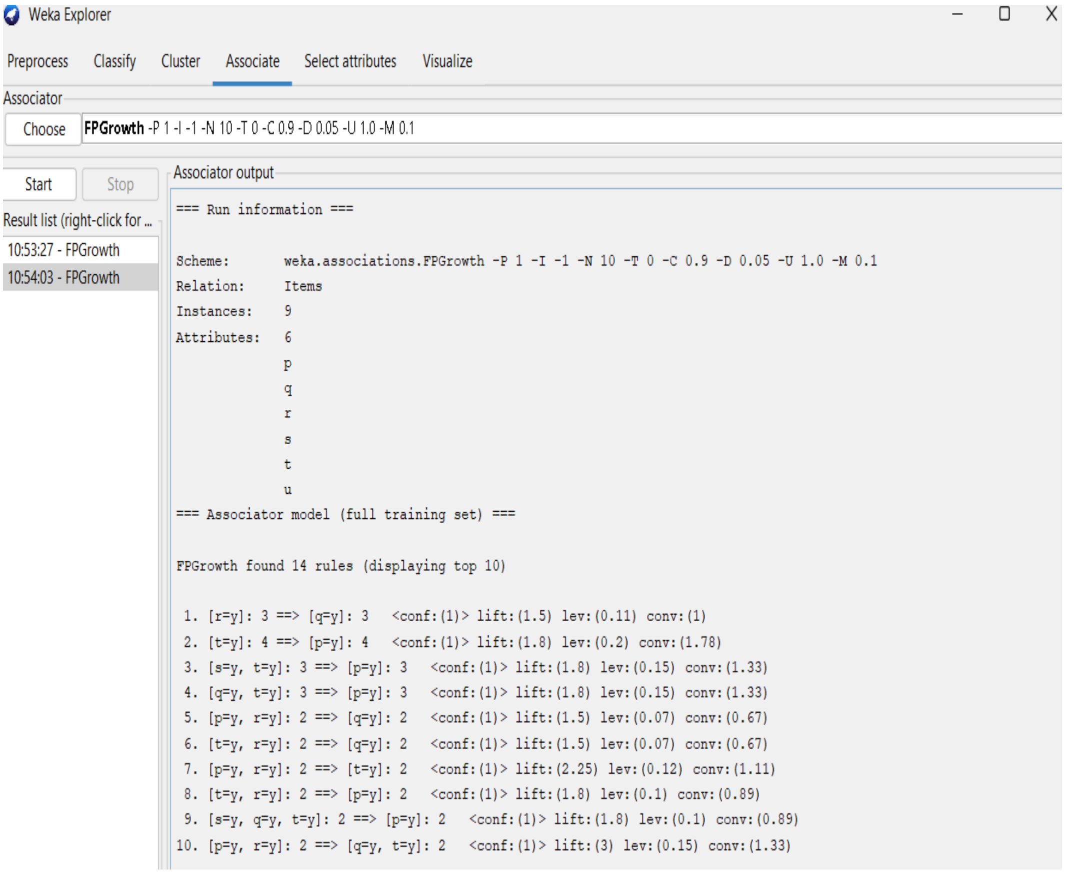


For Frequent Pattern Growth

Step 1: Select the appropriate dataset from the source file.



Step 2: Select FP Growth and with minimum support 2 and Click on start button and check for accuracy on output screen.



- Observations:
  FP-Growth works by **compressing the database** into an **FP-tree (Frequent Pattern Tree)**, which retains t he itemset association information but avoids generating candidate itemsets explicitly.

- The algorithm proceeds in two main major steps:

  1. **Building the FP-tree**: A tree structure that stores items in transactions in a compressed manner.

  2. **Mining the FP-tree recursively**: Extracts frequent itemsets via conditional pattern bases and condi tional FP-trees.

Conclusion:
FPGrowth is highly effective for frequent pattern mining in dense transactional datasets, offering **significant com putational savings** over candidate-generation methods. Its limitations occur mainly in sparse, highly diverse data sets or under extremely low minimum support thresholds. Proper preprocessing, item ordering, and careful memo ry management are crucial for maximizing FP-Growth performance.

Experiment No.:3

Demonstration of classification rule process on WEKA data-set using Naive Bayes algorithm.

Date of Performance:

Particulars	**Marks**
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
Signature of Course Teacher	

Objective: To build, train, and evaluate a Naive Bayes classifier on a given dataset in WEKA, in order to generate classification rules that can predict the class of new instances based on probabilistic relationships between attributes.

Theory:

Step 1: Step 1: Understanding Bayes' Theorem

Bayes' Theorem is the core of the Naive Bayes classifier:

$$P(C|X) = \frac{P(X|C)\cdot P(C)}{P(X)}$$

Where:

- $P(C|X)$ = Posterior probability of class C given features $X = (x_1, x_2, ..., x_n)$
- $P(C)$ = Prior probability of class C
- $P(X|C)$ = Likelihood of features X given class C
- $P(X)$ = Probability of features X (can be ignored for classification as it is constant for all classes)

Step 2: Principle of Bayesian Classification

- The Bayesian classifier predicts the class C that maximizes the posterior probability $P(C|X)$. In practice:
- $Predicted\ class = \arg arg_{max\ C} P(C|X)$
- Since $P(X)$ is constant for all classes, it is sufficient to maximize $P(X|C) \cdot P(C)$.

Step 3: Types of Bayesian Classifiers

1. Naive Bayes Classifier:

Assumes that attributes in the dataset are **independent** of each other given the class. Despite this simplifying a ssumption, it performs well in many real-world applications.

- The likelihood can be simplified as:

$$P(X|C) = P\left(x_1|C\right) \cdot P\left(x_2|C\right) \cdots P\left(x_n|C\right)$$

2.Gaussian, Multinomial, and Bernoulli Naive Bayes:

Different types of Naive Bayes classifiers are used depending on the data:

- **Gaussian:** for continuous data
- **Multinomial:** for discrete count data, like word frequencies
- **Bernoulli:** for binary features, e.g., email spam classification

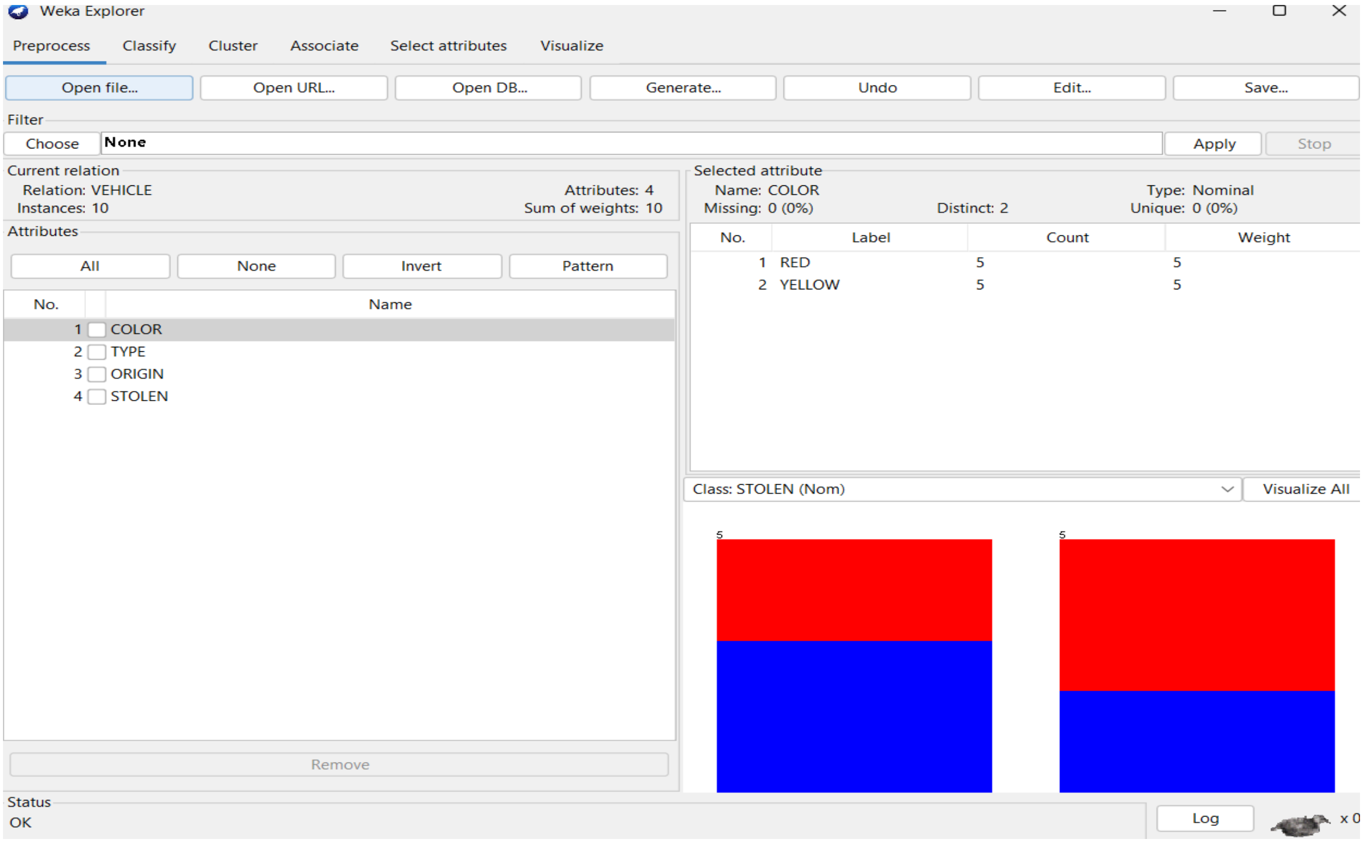
Step 4: Steps to Apply Bayesian Classification

- **Collect Data:** Gather the dataset with features and class labels.
- **Compute Priors:** Calculate prior probabilities $P(C)$ for each class.
- **Compute Likelihood:** Calculate likelihood $P(X|C)$ for each feature given the class.
- **Compute Posterior:** Apply Bayes' Theorem to compute $P(C|X)$ for each class.
- **Predict Class:** Choose the class with the maximum posterior probability.

Procedure:

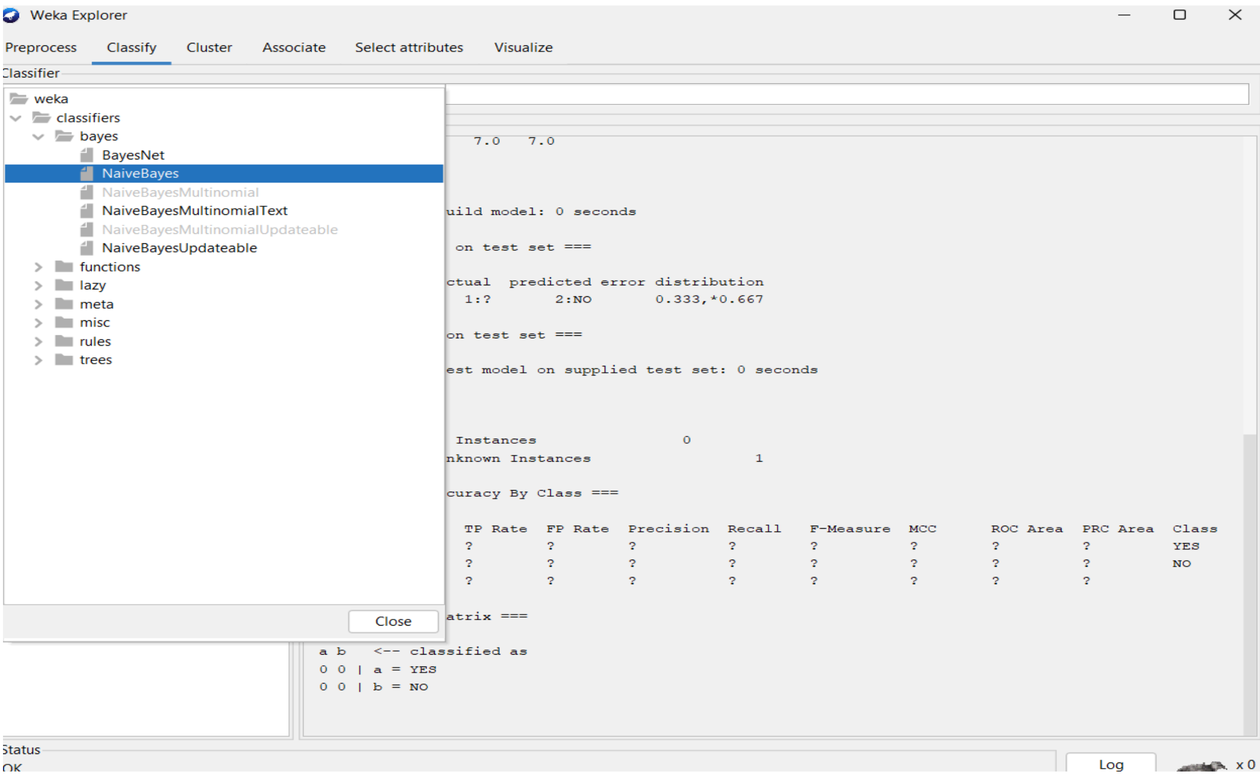
Step 1: Open WEKA and Load Data

1. Launch **WEKA** GUI Chooser.
2. Select **Explorer**.
3. Click **Open file** and load your dataset in **ARFF** or **CSV** format.
    - Example: weather nominal.arff (commonly used dataset for classification).



Step 2: Preprocess the Data

- The **Preprocess** tab allows you to:
    - Check **attributes** and **class distribution**.
    - Apply filters for missing values, normalization, or discretization if necessary.
- Make sure your **class attribute** is correctly set (usually the target variable).

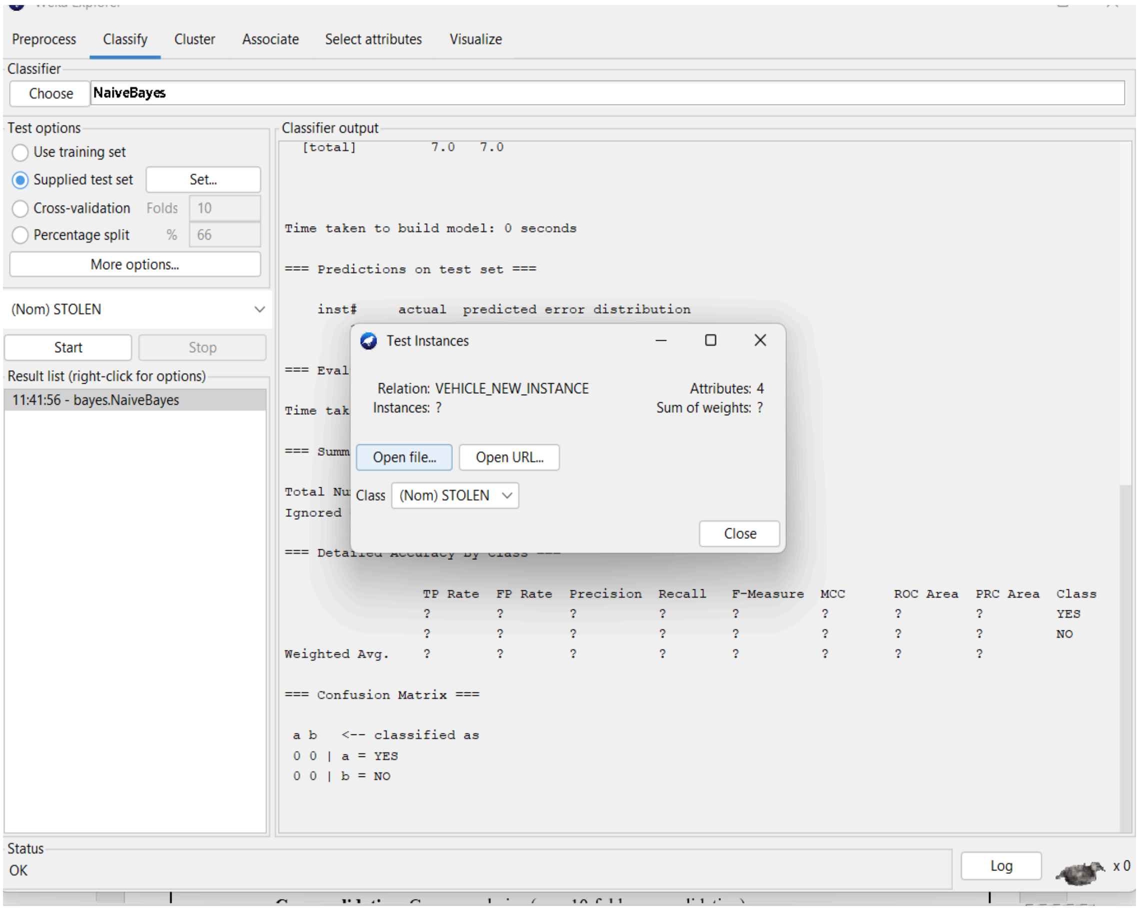


Step 3: Choose Classifier

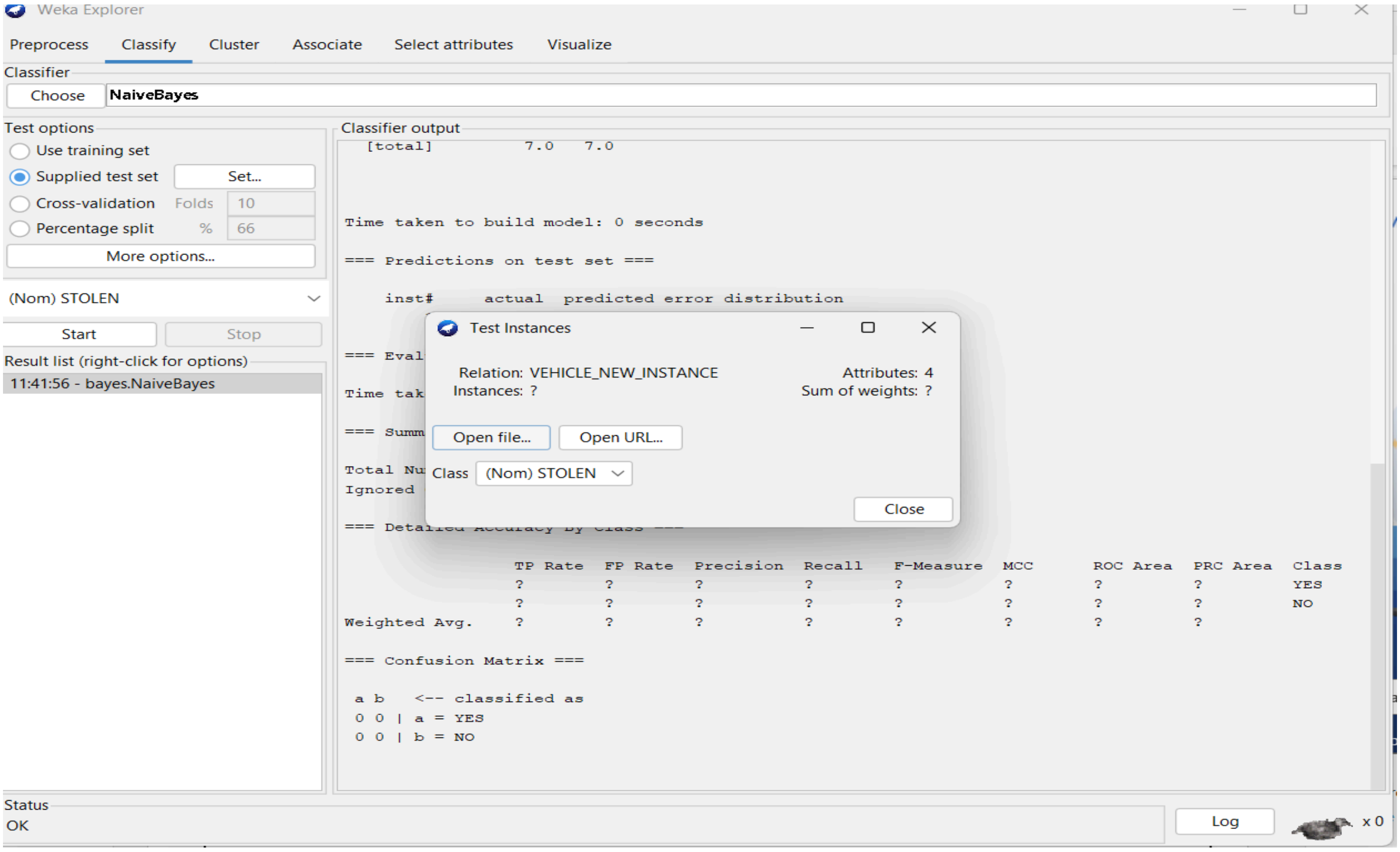
1. Go to the **Classify** tab.
2. Click **Choose** and select bayes -> NaiveBayes.
3. Optionally, click on the classifier to configure parameters:
    - NaiveBayes has minimal tuning; you can enable/disabling kernel estimation.

Step 4: Set Evaluation Method

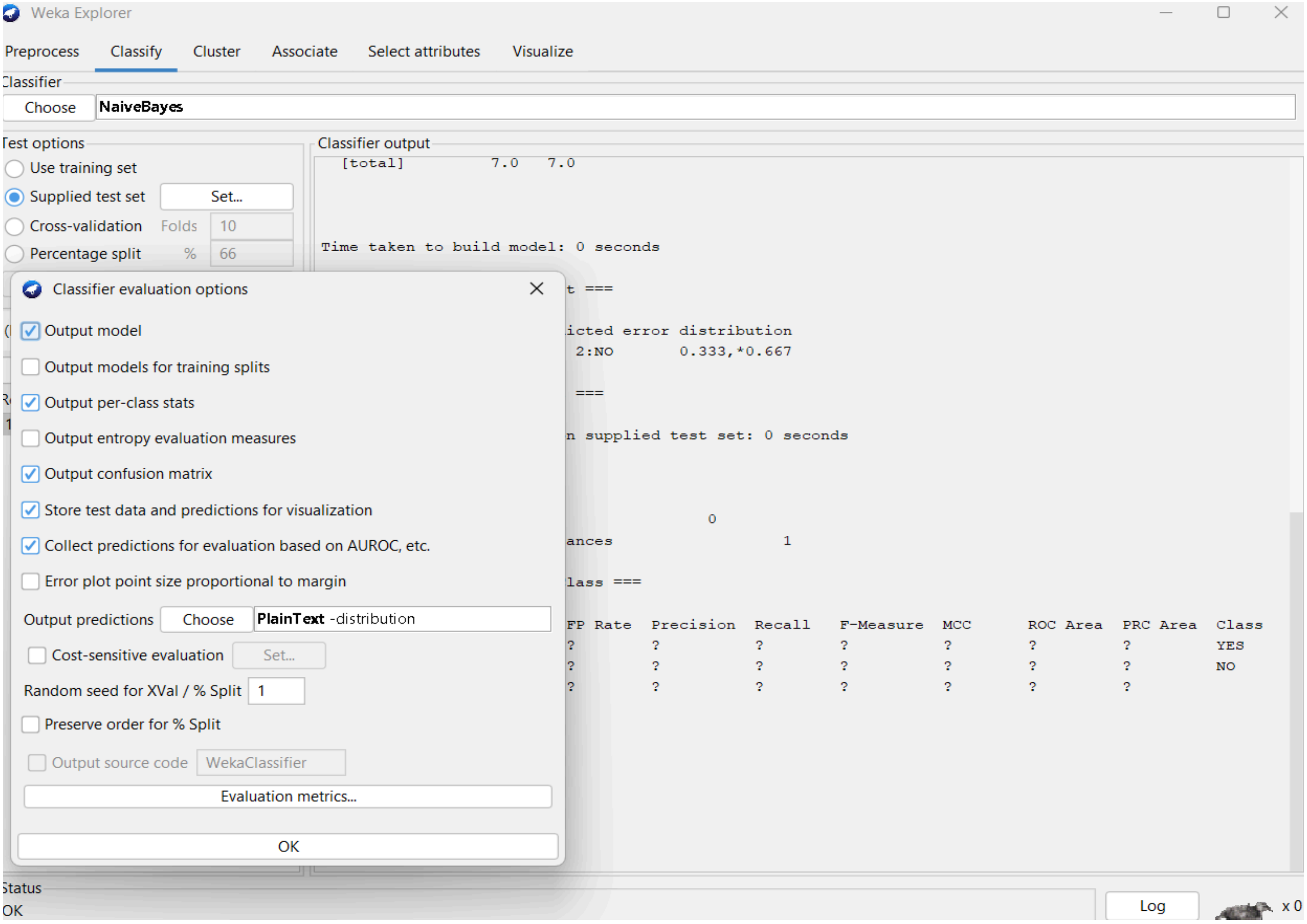
- Under **Test options**, choose how to evaluate the model:
  - **Use training set**: Tests on dataset used for training (not recommended for final evaluation).
  - **Cross-validation**: Common choice (e.g., 10-fold cross-validation).
  - **Percentage split**: Splits the dataset into train/test sets (e.g., 70/30).



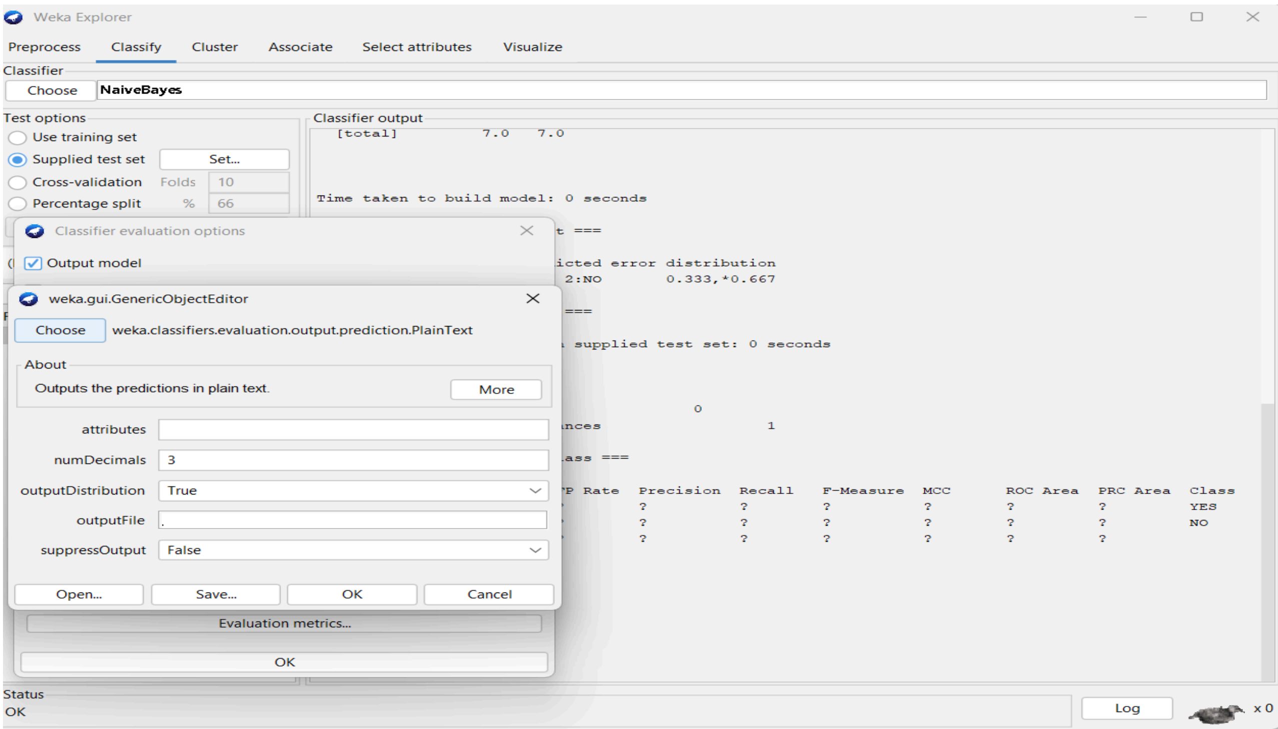
- **Supplied test set**: Test on a separate dataset file.



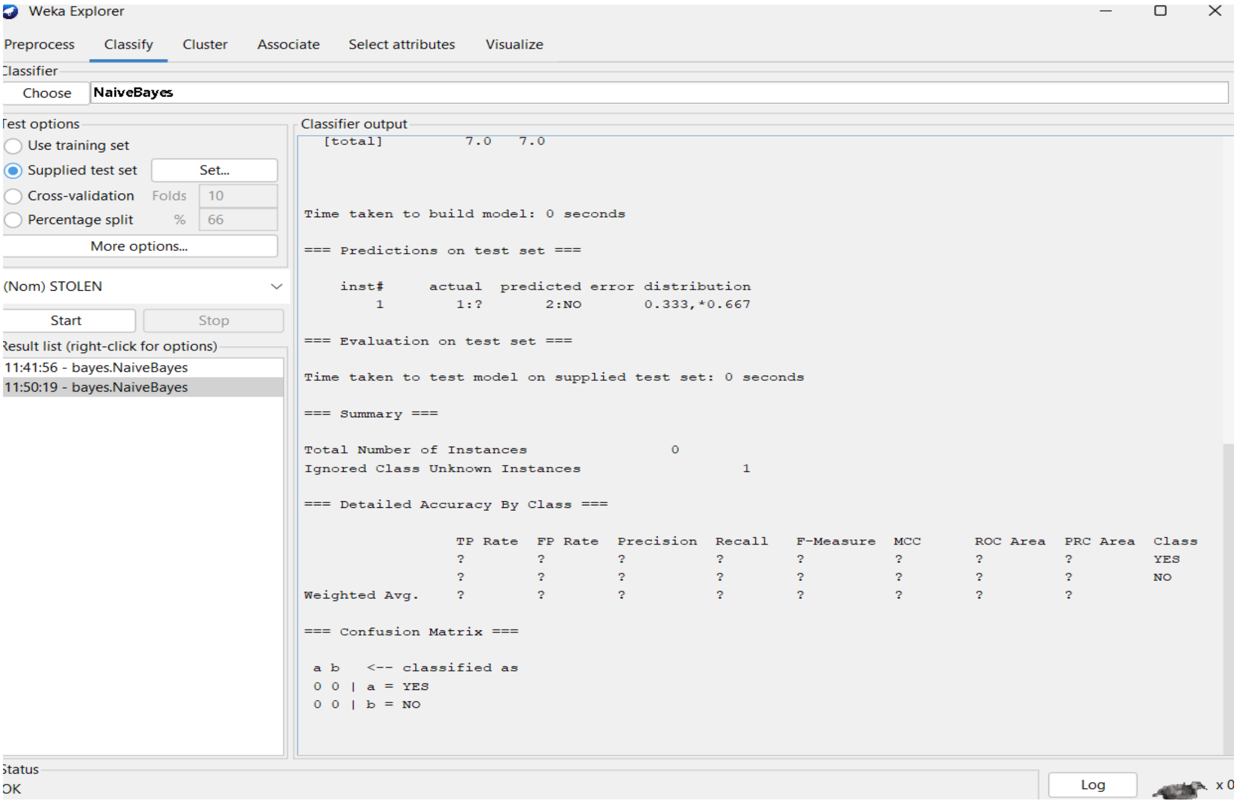
- Select Naïve Bayes Classifier evaluation options:



- Choose output prediction in plain text



- Click on start button to visualize the output screen as a result.



Observations:

It has been observed that the probability of selection of new vehicle with new instance is NO. Because distribution test set shows NO.

Results/Analysis:

Analysing Naive Bayes results in WEKA involves:

- Reviewing per-class metrics (Precision, Recall, F-Measure, ROC Area)
- Checking global performance (accuracy, Kappa, error rates)
- Examining the confusion matrix for misclassification patterns
- Considering adjustments in preprocessing, feature selection, or classifier parameters to optimize performan ce.

Conclusion:

Naive Bayes is best treated as a strong baseline classifier:

- Quick to implement and test for feasibility.
- Produces meaningful probabilistic outputs for decision-making.
- Works particularly well for **high-dimensional categorical data** and cases with limited training data .

Experiment No.:4

Implementation of OLAP operations

Date of Performance:

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
Signature of Course Teacher	

Objective:

The primary objective of implementing OLAP operations is to enable multidimensional analysis of large d atasets, supporting fast, flexible, and insightful decision-making by aggregating, drilling, slicing, dicing, an d rotating data across multiple perspectives

Theory: **1. Types of OLAP Servers**

Before implementing operations, you need to choose the type of OLAP architecture since it affects performance, storage, and query handling:

- **ROLAP (Relational OLAP):** Operates over relational databases; uses SQL for aggregation. Best for large, evolving datasets.
- **MOLAP (Multidimensional OLAP):** Pre-computes and stores data in multidimensional arrays; very fast f or queries but less flexible for schema changes.
- **HOLAP (Hybrid OLAP):** Combines ROLAP and MOLAP; aggregates stored in cubes, detailed data in re lational databases.
- **VOLAP (Virtual OLAP):** Provides a virtual cube layer on top of existing data sources without storing pre -aggregates.

2.OLAP Operations and Their Implementation

a) Roll-Up (Aggregation)

- **Purpose:** Moves up a hierarchy to summarize data at a higher level.
- **Implementation:** Aggregate measure values along a selected dimension using a **concept hierarchy**.
- **Example:** Sum sales of cities to compute country-level sales.

b) Drill-Down (Detailing)

- **Purpose:** Moves down a hierarchy to obtain more detailed data.
- **Implementation:** Expand dimensions or descend hierarchy levels, revealing finer granu larity.
- **Example:** Break annual sales into quarterly or monthly sales.

c) Slice

- **Purpose:** Selects a single value on one dimension, creating a sub-cube.
- **Implementation:** Apply a filter on one dimension.
- **Example:** Select sales data for Q1 across all products and regions.

d) Dice

- **Purpose:** Selects specific values on two or more dimensions, producing a sub-cube.
- **Implementation:** Apply multiple filters simultaneously.
- **Example:** Sales for Q1 and Q2 in North America for Electronics.

e) Pivot (Rotation)

- **Purpose:** Reorients the cube axes to offer alternative views.
- **Implementation:** Swap rows and columns or rearrange dimensions for visualization.
- **Example:** Switch from viewing sales by Region × Product to Product × Region.

Procedure:

Implementation Steps (OLAP Workflow)

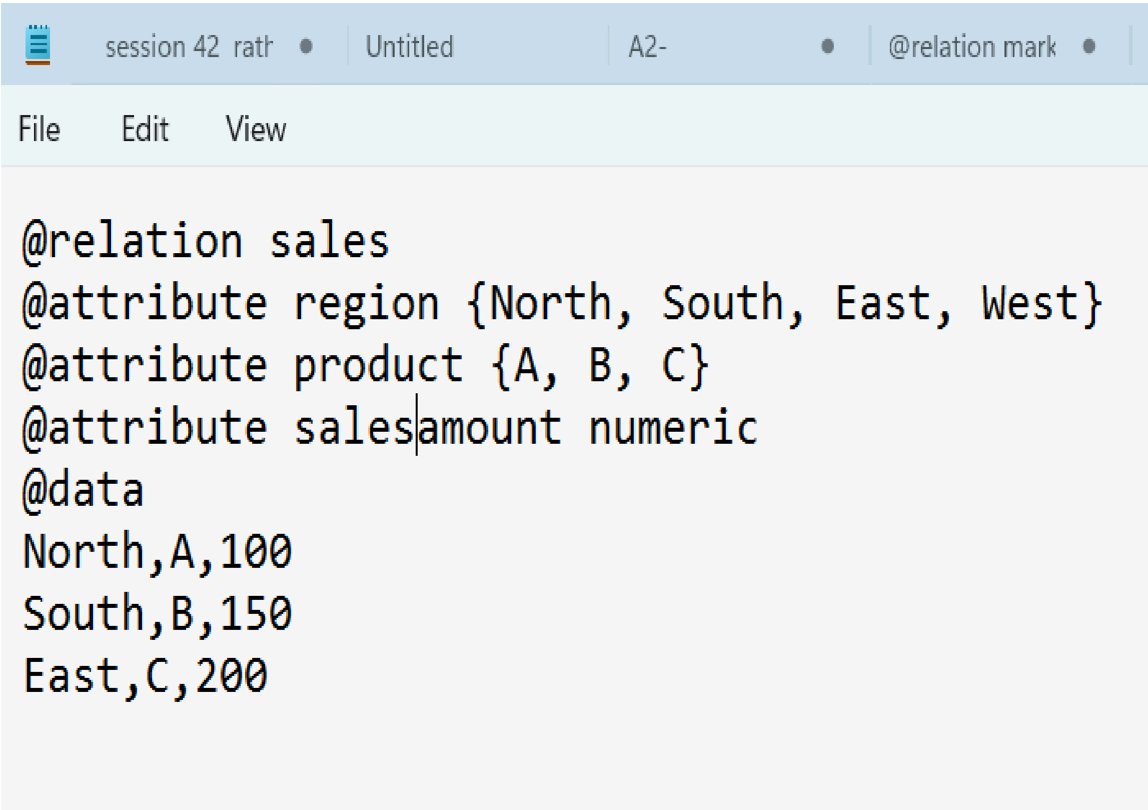
1. **Dimensional Modeling:** Design fact tables (measures) and dimension tables (attributes with hierarchies).
2. **Data Extraction:** Extract relevant data from OLTP systems or data warehouses.
3. **Data Loading (ETL/ELT):** Populate the OLAP system (ROLAP or MOLAP).
4. **Aggregation & Derivation:** Pre-compute common aggregates if necessary.
5. **Implement OLAP Operations:** Configure the OLAP engine or write SQL queries for roll-up, drill-down,

slice, dice, and pivot.

6. **Visualization & Access:** Provide BI tools (Excel, Tableau, Power BI) or dashboards for interactive analysi s.
7. **User Training:** Train analysts to navigate cubes and perform operations effectively.

1. Data Preparation

- Load your multidimensional dataset into Weka (CSV, ARFF formats are supported).
- Ensure your data includes categorical dimensions and numeric measures for aggregation.



```
@relation sales
@attribute region {North, South, East, West}
@attribute product {A, B, C}
@attribute salesamount numeric
@data
North,A,100
South,B,150
East,C,200
```

2. Roll-up and Drill-down

- **Roll-up:** Aggregate data at a higher level of a dimension.
- In Weka, use preprocessing GroupBy from the KnowledgeFlow or Filter options to summarize numeric attributes:
    - weka.filters.unsupervised.attribute.AddExpression can be used for computed fields.
    - Use weka.filters.unsupervised.attribute.GroupBy to aggregate by dimension (sum, mean, count).
- **Drill-down:** Filter records to a lower granularity using weka.filters.unsupervised.instance. SubsetByExpression:
    - Example: Viewing sales for a specific region and product category.

3. Slice and Dice Operations

- **Slice:** Select a particular dimension value to focus on, e.g., region = North. Use SubsetByExpression filter.
- **Dice:** Select a subcube over multiple dimensions, e.g., (region = North) AND (product = A). Again, apply SubsetByExpression with multiple conditions.

4. Pivoting / Cross-tabulation

- Weka's GUI has limited pivot features, but you can use weka.filters.unsupervised.attribute.Pivot to transfor m rows to columns by a dimension if needed. This creates multidimensional views similar to pivot tables in OLAP.

5. Aggregation and Analysis

- Use NumericToNominal and Discretize filters to group continuous values into bins (time periods, ranges, e tc.).
- Then apply GroupBy to compute aggregates: sum, mean, count, or custom expressions over measures.

Example Workflow in Weka GUI:

1. Load dataset (Explorer -> Open file).
2. Apply Preprocess -> Filter -> SubsetByExpression for slicing/dicing.
3. Use GroupBy filter to aggregate measures for roll-up.
4. Visualize aggregated results.
5. Repeat filtering for drill-down operations.

Observations:

- Enables detailed and aggregate data exploration,
- Supports dynamic reorientation and filtering of views,
- Improves efficiency of complex queries,
- Enhances decision-making through fast, actionable insights,

Conclusion:

Multidimensional Analysis: OLAP structures data in multidimensional cubes, also called hypercubes, where **di mensions** represent business aspects such as time, product, geography, or customer, and **measures** represent qua ntifiable metrics like sales, revenue, or profit.

Support for Business Intelligence and Decision-Making: By providing rapid, structured access to information, OLAP operations facilitate informed decision-maki ng, forecasting, and strategic planning. They allow organizations to compare metrics over time, across regions, or among product categories, ultimately enhancing **situational awareness and business performance**.

Support for Business Intelligence and Decision-Making: By providing rapid, structured access to information, OLAP operations facilitate informed decision making, forecasting, and strategic planning. They allow organizations to compare metrics over time, across region s, or among product categories, ultimately enhancing **situational awareness and business performance.**

Experiment No.:5

Demonstration of any ETL tool

Date of Performance:

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
Signature of Course Teacher	

Objective:

The primary objective of using Weka for ETL (Extract, Transform, Load) is to prepare raw data for machine learning by cleaning, formatting, and reducing it into a high-quality dataset.

Theory:

In the context of Weka, the theory of **ETL (Extract, Transform, Load)** centers on **Data Preprocessing**, which is the foundational step for any machine learning task. While traditional ETL is designed for data warehousing, Weka adapts these concepts to prepare raw datasets for algorithmic analysis.

1. Extract: Data Ingestion

The extraction phase in Weka involves retrieving raw data from diverse sources and moving it into a manageable environment.

- **Source Diversity**: Weka can extract data from local files (CSV, ARFF), web URLs, and relational databases via JDBC connectivity.
- **Staging**: In the Weka Explorer, once data is "opened," it is held in memory as a "working relation," which acts as a temporary staging area for further processing.

2. Transform: Data Preprocessing & Cleansing

This is the most critical phase where "Filters" are applied to ensure data quality and compatibility.

- **Data Cleansing**: Identifying and handling missing values (often marked with a '?' in Weka), removing outliers, and deduplicating records to prevent skewed results.
- **Format Standardization**: Converting data types, such as **Discretization** (turning numeric age into bins like 'youth' or 'senior') or **Normalization** (scaling values between 0 and 1).
- **Feature Engineering**: Using Attribute Selection to remove irrelevant or redundant fields (e.g., a unique 'ID' number), which simplifies the model and improves training efficiency.

3. Load: Result Persistence

The loading phase involves transferring the "cleaned" and "structured" data to a target system.

- **Saving to File**: The most common "load" in Weka is saving the preprocessed working relation into a new ARFF file for future model building.
- **Database Export**: Transformed data can also be written back to a database or used immediately by a learning algorithm in a unified pipeline.

Procedure:

1. **Launch WEKA and Open the Explorer:** Open the WEKA application and click the **Explorer** button in the main GUI Chooser window.
2. **Extract (Load) Data:** In the **Preprocess** tab, click the **Open file...** button to load a dataset (e.g., an ARFF or CSV file). This simulates the "Extract" phase, making the data accessible within WEKA.
3. **Transform Data (Apply Filters):** The Preprocess tab allows you to apply various filters which serve as data transformation methods.
4. **Select Attributes:** To remove unnecessary columns (attributes), use the "Remove" filter under weka.filters.unsupervised.attribute. Click Choose, navigate to unsupervised -> attribute -> Remove, then click on the filter name to configure which attributes to delete.
5. **Handle Missing Values:** The ReplaceMissingValues filter can automatically handle missing data points by replacing them (e.g., with the mean or mode of the attribute).
6. **Standardization/Normalization:** Filters like Standardize or Normalize can be used to format data consistently, which is a key part of the transform step.
7. **Apply the filter:** After configuring a filter, click the **Apply** button to execute the transformation on the loaded data.
8. **Load Data (Save the Transformed File):** Once the transformations are complete, you can save the

modified dataset in a new file (e.g., as a new ARFF file or CSV file) by clicking the **Save...** button. This simulates loading the transformed data into a new location.

Observations:

1. The "Working Relation" (Memory-Based)

Unlike enterprise ETL tools that stream data, Weka typically loads the entire dataset into **RAM** first.

- **Observation:** You will notice that the "Extract" phase is limited by your computer's memory. For very large datasets, Weka may lag or throw an "Out of Memory" error unless you use the *InstancesRepository* or command-line interface.

2. Immediate Visual Feedback (The Histogram)

In the **Preprocess** tab, every "Transformation" (filter) you apply triggers an instant update to the attribute visualisations.

- **Observation:** If you apply a **Discretize** filter to a numeric column like "Age," you will see the smooth distribution curve turn into a series of distinct bars (bins). This allows for real-time validation of the transform step.

3. Impact of Metadata

Weka is extremely sensitive to **Attribute Types** (Nominal vs. Numeric).

- **Observation:** If your "Extract" phase incorrectly identifies a category as a number, many "Transform" filters will be greyed out. You'll often observe that a mandatory first step is using the NumericToNominal filter to fix the data schema.

4. The "Undo" Safety Net

Weka maintains a history of transformations during a session.

- **Observation:** Because ETL is often trial-and-error, the **Undo** button acts as a version control for your "Load" state. You can revert a bad transformation (like an over-aggressive Remove filter) without re-extracting the original file.

5. Deterministic vs. Stochastic Transforms

Some transformations in Weka (like Randomize or Resample) change the data order or composition based on a "Seed" value.

- **Observation:** If you "Load" the same raw data twice but get different model results, it's usually because a stochastic (randomized) transform was applied during the ETL pipeline.

6. Knowledge Flow "Nodes"

In the **Knowledge Flow** interface, you can observe the "Load" happening in parallel.

- **Observation:** You can connect one "Transform" (filter) to multiple "Load" targets (e.g., a CSV Saver and a Visualization component) simultaneously, showing how data branches out after cleaning.

Results/Analysis:

1. Extraction Results (Ingestion)

When you first load a file (Extract), Weka provides an immediate summary of the raw data:

- **Instance/Attribute Count**: Confirms all records and columns were successfully read from the source (e.g., CSV or SQL).
- **Type Identification**: Displays whether Weka correctly identified attributes as **Nominal** (categorical) or **Numeric**.
- **Missing Values**: Shows the exact count and percentage of '?' entries, highlighting "dirty" data that needs transformation.

2. Transformation Results (Preprocessing)

As you apply filters, you can analyze the change in data distribution:

- **Cleaning Results**: After applying the ReplaceMissingValues filter, the "Missing" count for an attribute should drop to **0%**. You will observe that the previously missing entries are now filled with the attribute's **mean** (for numeric) or **mode** (for nominal).
- **Normalization/Standardization Results**: Before transformation, a "Price" attribute might range from 10 to 10,000. After applying a Normalize filter, the result window will show a new range (typically **0.0 to 1.0**), which is critical for distance-based algorithms.
- **Attribute Selection Results**: If you use a "Wrapper" or "Information Gain" evaluator, Weka outputs a ranked list of attributes. Analysis of these results often shows that a significant number of attributes (e.g., ID numbers or redundant sensors) can be removed without losing predictive power.

3. Load & Validation Analysis

The final ETL stage involves verifying that the transformed data is "better" for its intended purpose:

- **Class Separability**: By clicking **Visualize All**, you can analyze how well the transformed features separate the target classes (indicated by color clusters). Cleaned data often shows clearer boundaries between groups.
- **Accuracy Uplift**: The most common analysis is comparing model accuracy before and after ETL. For example, a study using a bank dataset found that cleaning missing and noisy data increased classification accuracy from **78.82% to 98.27%**.
- **Reduced Complexity**: Successful ETL results in fewer attributes and instances, which is observed as faster training times and a more interpretable model (e.g., a smaller J48 Decision Tree)

Conclusion:

a demonstration of **ETL in Weka** proves that while the tool is built for machine learning, its preprocessing power makes it an effective "lightweight" ETL engine for data science proj

Experiment No.:6

Write a program of Apriori algorithm using any programming language.

Date of Performance:

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	

Objective:

The primary **objective** of the Apriori algorithm is **Association Rule Mining**—discovering interesting relationships, patterns, or co-occurrences between items in large datasets

Theory:

The theory of the **Apriori Algorithm** is based on the principle of **Association Rule Mining**, a data mining technique used to identify frequent patterns and relationships in transactional databases. Introduced in 1994 by Rakesh Agrawal and Ramakrishnan Srikant, it is famously used for **Market Basket Analysis** to understand which items customers frequently purchase together

Procedure:

The **Apriori Algorithm** follows a iterative, "level-wise" procedure. It starts by finding frequent individual items and uses them as seeds to find larger frequent itemsets.

Step-by-Step Procedure

1. **Set the Thresholds**:
   Define a **Minimum Support** (e.g., 50%) and **Minimum Confidence** to filter out rare or weak associations.
2. **Generate 1-Itemset Candidates ()**:
   Scan the entire database to count the occurrences of every single unique item.
3. **Filter/Prune ()**:
   Compare the count of each item against the Minimum Support. Keep only those that meet or exceed the threshold; discard the rest.
4. **Self-Join to Generate Candidates ()**:
   Take the frequent itemsets from the previous step () and join them with each other to create larger combinations (e.g., combining {Bread} and {Milk} to form {Bread, Milk}).
5. **Apply the Apriori Property (Pruning)**:
   Before scanning the database again, check the subsets of the new candidates. If any subset of a candidate was already found to be infrequent in the previous step, **discard** that candidate immediately without checking the data.
6. **Repeat Until Completion**:
   Continue scanning the database, counting support, and generating larger sets until no more frequent itemsets can be formed.
7. **Generate Association Rules**:
   Once all frequent itemsets are found, extract rules (e.g.,) and calculate their **Confidence** and **Lift** to determine which relationships are actually significant.

Program using python

```
from itertools import combinations

def get_support(transactions, itemset):
    """Calculates the support of a given itemset."""
    count = 0
```

```
    for tx in transactions:
        if set(itemset).issubset(set(tx)):
            count += 1
    return count / len(transactions)



def apriori(transactions, min_support):
    """Core Apriori logic to find frequent itemsets."""
    # Step 1: Find unique items and create initial 1-itemsets
    unique_items = sorted(list(set(item for tx in transactions for item in tx)))
    current_candidates = [[item] for item in unique_items]

    all_frequent_itemsets = []
    k = 1

    while current_candidates:
        # Step 2: Filter candidates by minimum support (Pruning)
        frequent_itemsets = []
        for itemset in current_candidates:
            support = get_support(transactions, itemset)
            if support >= min_support:
                frequent_itemsets.append((tuple(itemset), support))

        # If no itemsets meet support, stop
        if not frequent_itemsets:
            break

        all_frequent_itemsets.extend(frequent_itemsets)

        # Step 3: Generate next level candidates (C_k+1)
        found_items = sorted(list(set(item for itemset, sup in frequent_itemsets for item in itemset)))
        current_candidates = list(combinations(found_items, k + 1))
        k += 1

    return all_frequent_itemsets

# --- Example Usage ---
# A small transactional dataset
dataset = [
    ['Milk', 'Bread', 'Wheat'],
    ['Bread', 'Eggs'],
    ['Milk', 'Bread', 'Eggs'],
    ['Milk', 'Eggs'],
```

```
    ['Bread', 'Wheat']
]

# Set minimum support to 40%
min_sup = 0.4
frequent_patterns = apriori(dataset, min_sup)

print(f"Frequent Itemsets (min_support={min_sup}):")
for itemset, support in frequent_patterns:
    print(f"Itemset: {list(itemset)}, Support: {support:.2f}")
```

Observations:
You will see that the **Support** for {A, B} is the same as {B, A}. However, the **Confidence** of
is often very different from , showing that the direction of the relationship matters in the "Load" or "Analysis" phase.

Results/Analysis:

1. Frequent Itemset Results
Based on a dataset of 5 transactions and a **Minimum Support of 40%** (minimum count of 2), the algorithm produces the following results:

- **Frequent 1-Itemsets ():**
    - **Bread**: 80% (Found in 4/5 transactions)
    - **Eggs**: 60% (Found in 3/5 transactions)
    - **Milk**: 60% (Found in 3/5 transactions)
    - **Wheat**: 40% (Found in 2/5 transactions)
- **Frequent 2-Itemsets ():**
    - **{Bread, Eggs}**: 40%
    - **{Bread, Milk}**: 40%
    - **{Bread, Wheat}**: 40%
    - **{Eggs, Milk}**: 40%
- **Frequent 3-Itemsets ():**
    - **None**: No triplet appears in at least 2 transactions. The algorithm terminates here as no further extensions are possible.

2. Rule Analysis and Metrics
To interpret these results, we analyse specific association rules (e.g.,) using standard metrics:

- **Support Analysis**: Indicates the overall popularity of a combination. For example, the pair **{Bread, Milk}** has a support of 0.4, meaning it appeared in 40% of all transactions.
- **Confidence Analysis**: Measures how often the items appear together relative to the first item.
    - **Rule $\{Milk\} \rightarrow \{Bread\}$**: The confidence is $Support(Milk, Bread)/Support(Milk) = 0.4/0.6 \approx 66.7\%$.

  This suggests that 66.7% of customers who bought Milk also purchased Bread.
- **Lift Analysis**: Determines if the association is stronger than random chance.

- If **Lift > 1**, there is a positive correlation; if **Lift = 1**, the items are independent.
- For the rule , the Lift is

- **Analysis**: Since the lift is less than 1, Milk and Bread are actually **less likely** to be bought together than if they were independent, despite the high.

Conclusion:
The **Apriori Algorithm** serves as a foundational technique for **Association Rule Mining**, effectively turning raw transactional data into actionable behavioral insight

Experiment No.:7

Case Study on Text Mining or any commercial application

Date of Performance:

Particulars	**Marks**
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
SignatureofCourse Teacher	

Objective: Text mining is a specialized process that transforms unstructured string data into a structured numerical format using the **Vector Space Model**. This allows standard machine learning algorithms to "read" and classify text.

Theory:

Case Study: Sentiment Analysis of Customer Reviews

A common commercial application in Weka is categorizing customer feedback (e.g., from Traveloka or Twitter) into Positive or Negative sentiments.

- TextDirectoryLoader: Used to load text files from specific subdirectories (where the folder name acts as the class label) directly into Weka.
- ARFF Format: Alternatively, data is loaded from an .arff file where the text is stored in a string attribute.

2. Transformation (The StringToWordVector Filter)

The core of text mining in Weka is the StringToWordVector filter. It performs several critical sub-steps:

- Tokenization: Breaking sentences into individual words (features).
- Case Normalization: Converting all text to lowercase to ensure "Apple" and "apple" are treated as the same feature.
- Stopword Removal: Deleting common words (e.g., "the", "is") that carry no predictive value.
- TF-IDF Weighting: Optionally calculating the "Importance" of a word rather than just its count.

Analysis & Results

In comparative studies using Weka for sentiment classification:

- Support Vector Machines (SVM): Frequently yields the highest accuracy, often reaching 91%–94% for product or movie reviews.
- Naive Bayes: A popular baseline that is computationally fast and performs well on small to medium text datasets, typically achieving around 82% accuracy.
- J48 (Decision Tree): Useful for visualizing which specific words (e.g., "horrible" or "excellent") are the primary "deciders" for a sentiment category.

Observations in Weka Text Mining

- Attribute Explosion: After applying StringToWordVector, a single string attribute might be replaced by thousands of numeric word attributes.
- Dimensionality Reduction: Users often use the word to Keep setting to limit the vocabulary to the most frequent 500–1000 terms to avoid memory crashes.
- Class Attribute: Many filters move the "Class" label to the beginning of the dataset, which may require Reorder filter to fix before classification.

Results and Analysis:

In Weka, the Results and Analysis of a text mining task are viewed in the Classifier Output panel after a model is trained. These results quantify how effectively the text (now a numerical vector) can be categorized into classes like "Positive" or "Negative."

1. Key Performance Metrics

Weka provides a comprehensive statistical breakdown to analyze model performance:

- Accuracy (Correctly Classified Instances): The percentage of text samples assigned to the right category. In sentiment analysis studies using Weka, SVM often achieves high accuracy (around 91%), while Decision Trees (J48) may reach approximately 87.5%.
- Precision: Measures exactness—the proportion of instances flagged as a class (e.g., "Positive") that actually belong to it.
- Recall (TP Rate): Measures completeness—how many of the actual "Positive" reviews the model successfully found.

- F-Measure: A harmonic mean of Precision and Recall, providing a single score to balance both
- Kappa Statistic: A chance-corrected measure; a value > 0 indicates the model is performing better than random guessing.

2. The Confusion Matrix (Error Analysis)

The confusion matrix is the primary tool for deep analysis of where the model is "confused".

- Analysis: If the matrix shows many "Positive" reviews being classified as "Negative," you might need to adjust your StringToWordVector settings—for example, by including N-grams to capture phrases like "not good" instead of just the word "good".

3. Case Study Findings: Sentiment Analysis

In commercial case studies (e.g., analyzing product reviews or Twitter data):

- Algorithm Comparison: Support Vector Machines (SMO/SVM) typically outperform Naive Bayes and J48 on text data because they handle high-dimensional word vectors more efficiently.
- Attribute Importance: By using Weka's visualization tools, you can analyze which words (attributes) have the highest "Information Gain," identifying the specific terms that most strongly signal a particular sentiment.
- Efficiency: While more complex models like Random Forest or Neural Networks may yield slightly higher accuracy, they are often observed to be significantly more time-consuming to train on text vectors compared to Naive Bayes.

4. Visual Analysis

Weka allows you to visualize the ROC Area (Area Under the Curve). A curve pushing toward the upper-left corner indicates a high-quality model that maintains a high true-positive rate while keeping false positives lo